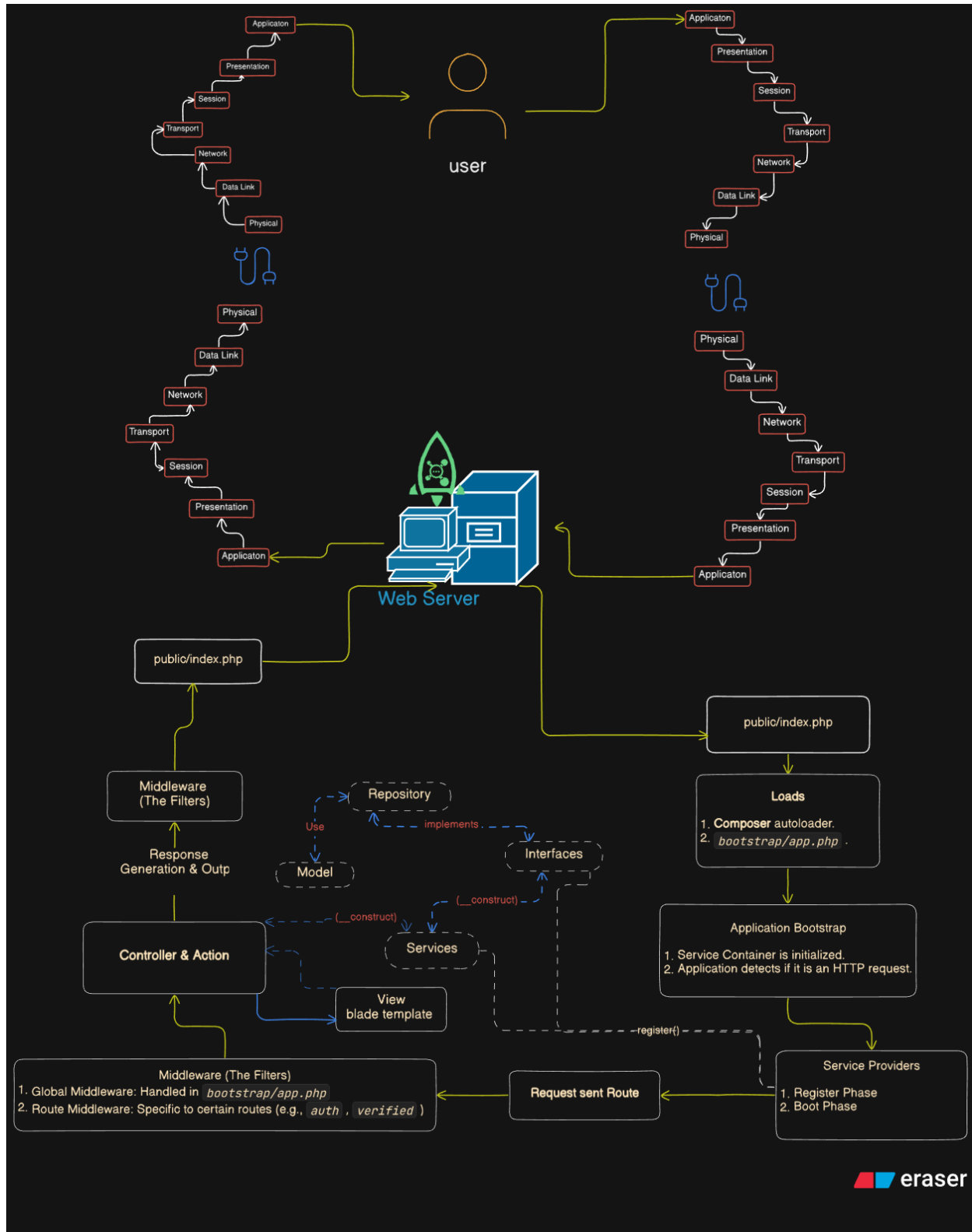


Laravel



সূচিপত্র — Table of Contents

১. Laravel Request Lifecycle

একটি HTTP Request Laravel অ্যাপ্লিকেশনে ঢোকার পর থেকে Response বের হওয়া পর্যন্ত যে পথ পাড়ি দেয়, তাকে Request Lifecycle বলে। নিচের diagram-এ সম্পূর্ণ flow দেখানো হয়েছে:

#	Layer / Component	Description
1	User Browser	HTTP Request sent
2	public/index.php	Entry point — loads Composer autoloader & bootstrap/app.php
3	Service Providers	register() → Boot Phase — binds services to Container
4	Middleware (Before)	Global & Route Middleware filters request (auth, CSRF, etc.)
5	Router	Matches URL to Controller@Action or Closure
6	Controller & Action	Business logic — uses Models, Services, Repositories
7	Eloquent Model	Database interaction via ORM
8	View / Blade Template	Renders HTML response
9	Middleware (After)	After middleware can modify response
10	HTTP Response	Final response sent back to browser

OSI Model ও Web Request

User Browser থেকে Web Server পর্যন্ত request যাওয়ার সময় OSI Model-এর 7টি layer পার হয়: Application → Presentation → Session → Transport → Network → Data Link → Physical। একইভাবে response ফেরত আসে।

Application Bootstrap

public/index.php-তে request আসলে দুটি কাজ হয়:

- Composer autoloader load হয়
- bootstrap/app.php লোড হয় — Service Container initialize হয়

২. Service Container ও Service Provider

Service Container (সার্ভিস কন্টেইনার)

সংজ্ঞা

Service Container হলো Laravel-এর IoC (Inversion of Control) Container। এটি Class Dependencies ম্যানেজ করে এবং Dependency Injection সম্পন্ন করে। যখনই কোনো ক্লাসের object দরকার হয়, Container নিজে থেকে dependencies সমাধান করে ready object return করে।

```
// Service Container-এ Bind করা
app()->bind(PaymentGateway::class, StripeGateway::class);

// Singleton — একটিই instance থাকবে
app()->singleton(DatabaseConnection::class, function($app) {
    return new DatabaseConnection(config('database'));
});

// Resolve করা
$gateway = app(PaymentGateway::class);
```

Service Provider (সার্ভিস প্রোভাইডার)

Service Provider হলো Laravel Application-এর Bootstrapping-এর মূল কেন্দ্র। এর প্রধান কাজ Services বা Classes-কে Service Container-এ Bind করা।

Method	কখন কল হয়	কী কাজ করে
register()	সব Provider load হওয়ার আগে	Container-এ service bind করে — শুধু বলে দেয় কী কী service আছে
boot()	সব Provider register হওয়ার পরে	Service কীভাবে ব্যবহার হবে তা define করে — routes, views, events

```
class AppServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // শুধু binding এখানে
        $this->app->bind(PaymentInterface::class, StripeService::class);
    }

    public function boot(): void
    {
        // সব provider ready — এখন logic লেখা যাবে
        View::share('appName', config('app.name'));
    }
}
```

}

৩. Laravel Facade

Facade কী?

সংজ্ঞা

Facade হলো Service Container-এ থাকা কোনো class-এর Static Interface বা Static Proxy। এটি non-static method-কে static হিসেবে call করার সুবিধা দেয়। সহজ syntax + testability (Mock করা যায়) — দুটি সমস্যা একসাথে সমাধান করে।

Facade-এর Internal Mechanism

Cache::get('key') call করলে ভেতরে যা হয়:

Step	Action	Detail
1	Cache::get('key') call	Static method কল করা হয়
2	__callStatic() triggered	PHP magic method সক্রিয় হয়
3	getFacadeAccessor() checked	Facade class-এ 'cache' string return হয়
4	Service Container lookup	Container থেকে real object খুঁজে বের করা হয়
5	Real method executes	আসল CacheManager-এর get() method রান হয়

```
// Facade ব্যবহার (সহজ)
Cache::put('key', 'value', 3600);
$value = Cache::get('key');

// Dependency Injection দিয়ে (verbose)
public function __construct(private CacheManager $cache) {}
$this->cache->put('key', 'value', 3600);
```

8. Observer ও Helper

Observer (মডেল স্পেসিফিক)

Observer একটি নির্দিষ্ট Model-এর উপর কাজ করে। Model-এর data তৈরি, আপডেট বা ডিলিট হলে অটোমেটিক trigger হয়।

বৈশিষ্ট্য	Observer	Global Scope
প্রয়োজ্য	নির্দিষ্ট একটি Model	সব Model বা বড় অংশ
কাজের ধরন	Event-based (Created, Updated, Deleted)	Query-based (Select, Filter)
ব্যবহার	Model lifecycle hook	Global query constraint

```
// Observer তৈরি
php artisan make:observer OrderObserver --model=Order

class OrderObserver
{
    public function created(Order $order): void
    {
        // Order তৈরি হলে automatically inventory update
        Inventory::decrease($order->items);
    }

    public function updated(Order $order): void { ... }
    public function deleted(Order $order): void { ... }
}
```

Helper Functions

Laravel Helper

Helper হলো Global Functions — project-এর যেকোনো জায়গা থেকে সরাসরি ব্যবহার করা যায়।

Controller, Model, Blade, Middleware — সব জায়গায় new ClassName() বা use Namespace ছাড়াই ব্যবহার করা যায়।

উদাহরণ: route(), asset(), dd(), collect(), now(), auth(), config()

৫. Middleware, Authentication ও Authorization

Middleware কী?

Middleware হলো HTTP Request এবং Response-এর মাঝে একটি Filter বা Layer। Request process হওয়ার আগে বা Response পাঠানোর আগে নির্দিষ্ট কাজ বা যাচাইকরণ করে।

ধরন	বিবরণ	উদাহরণ
Global Middleware	সব request-এ apply হয়	bootstrap/app.php-তে handle
Route Middleware	নির্দিষ্ট route-এ apply হয়	auth, verified, throttle:60,1
Before Middleware	Request process-এর আগে চলে	Auth check, CSRF verify
After Middleware	Response পাঠানোর আগে চলে	Response modify, logging

Authentication vs Authorization

বিষয়	Authentication (অথেনটিকেশন)	Authorization (অথরাইজেশন)
প্রশ্ন	ইউজার কে? (Who)	ইউজার কী করতে পারবে? (What)
কাজ	পরিচয় যাচাই	অনুমতি যাচাই
উদাহরণ	Username/Password, OTP login	Admin delete করতে পারে, User শুধু দেখতে পারে
Laravel Tool	Sanctum, Passport	Gate, Policy

Gate vs Policy

বিষয়	Gate	Policy
ধরন	Closure-based	Class-based
ব্যবহার	ছোট, সিম্পল কাজ	বড়, complex project
Model সংযোগ	Model-এর সাথে যুক্ত নয়	নির্দিষ্ট Model-এর সাথে যুক্ত
Define করা হয়	AppServiceProvider-এ	আলাদা Policy class-এ
উদাহরণ	Admin panel access check	User শুধু নিজের post edit করতে পারবে

CSRF Token ছাড়া নির্দিষ্ট Route Exclude

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware): void {
    $middleware->validateCsrfTokens(except: [
```

```
'stripe/*',          // Stripe payment সব URL
'payment-callback', // নির্দিষ্ট একটি পেজ
'webhook/handler',  // External service থেকে আসা request
]);
})
```

৬. Event ও Listener

Event-Listener Pattern

কেন ব্যবহার করি?

ERP-তে একটি কাজের পরিপ্রেক্ষিতে অনেকগুলো sub-task automatically সম্পন্ন করতে হয়।

Code clean ও maintainable রাখার জন্য Event-Listener pattern ব্যবহার করা হয়।

Event fire হলে registered সব Listener automatically execute হয় — decoupled architecture।

```
// EventServiceProvider.php
protected $listen = [
    OrderConfirmed::class => [
        UpdateInventoryAfterOrder::class, // Inventory কমানো
        CreateAccountingEntry::class,     // Accounting entry
        SendOrderNotification::class,     // Notification পাঠানো
    ],
];

// Event fire করা
event(new OrderConfirmed($order));

// অথবা Model-এ
OrderConfirmed::dispatch($order);
```

৭. Eloquent Relationships ও N+1 Problem

N+1 Problem ও Eager Loading

N+1 Problem কী?

Main model-এর data আনার পর loop-এর ভেতরে related data access করলে প্রতিবার নতুন query চলে।

১০০টি User থাকলে → ১টি User query + ১০০টি Post query = ১০১টি query! এটাই N+1 Problem।

Method	ব্যবহার	Query সংখ্যা	কখন ব্যবহার করবেন
with()	User::with('posts')->get()	মাত্র ২টি	Main query-র সময়ই related data দরকার
load()	\$users->load('posts')	পরে আরও কিছু query	Already retrieved data-তে পরে related data লাগলে
withCount()	User::withCount('posts')->get()	১টি (subquery)	Actual data নয়, শুধু count দরকার হলে

HasManyThrough Relationship

দূর সম্পর্কের relation। যেমন: Country → User → Post। Country থেকে সরাসরি সব Post পেতে।

```
// Country Model
public function posts(): HasManyThrough
{
    return $this->hasManyThrough(Post::class, User::class)
        ->whereNull('users.deleted_at'); // Soft delete handle
}

// ব্যবহার
$bdPosts = Country::find(1)->posts;
```

Polymorphic Relations

Type	ব্যবহার	Table Structure
MorphMany	Post ও Video উভয়ই Comment পাবে	comments: commentable_id, commentable_type
MorphToMany	Post ও Video উভয়ে অনেক Tag থাকতে পারে	tags + taggables pivot table

Caching: File vs Redis

বৈশিষ্ট্য	File Caching	Redis Caching
Storage	Hard Disk (SSD/HDD)	RAM (In-memory)
Speed	তুলনামূলক কম (Disk I/O)	অত্যন্ত দ্রুত
Cache Tags	সাপোর্ট করে না	খুব ভালো সাপোর্ট করে
Scalability	এক সার্ভারে সীমাবদ্ধ	Multi-server shared cache সহজ
Setup	কোনো বাড়তি setup লাগে না	আলাদা Redis server install করতে হয়

৮. Job Queue ও Task Scheduling

Job Queue কী?

PHP Synchronous language। ভারী কাজ (email, image processing) user-কে লাইনে দাঁড় করিয়ে না করে Background-এ পাঠিয়ে দেওয়া হয় Queue-তে।

Queue Workflow

⚙ Dispatch Job Controller calls <code>dispatch(\$job)</code>	⚙ Queue Driver Job stored in Redis / Database / SQS	⚙ Queue Worker <code>php artisan queue:work</code> processes jobs	⚙ Execute Job logic runs (email, image, PDF...)
---	--	--	--

```
// 1. Job তৈরি
php artisan make:job SendWelcomeEmail

// 2. Job Dispatch করা
SendWelcomeEmail::dispatch($user);
SendWelcomeEmail::dispatch($user)->delay(now()->addMinutes(10));

// 3. Worker চালু করা
php artisan queue:work --queue=emails
```

Queue Chain vs Queue Batch

বৈশিষ্ট্য	Queue Chain	Queue Batch
কাজের ধরন	Serial — একটার পর একটা	Parallel — সব একসাথে
নির্ভরশীলতা	পরবর্তী job আগেরটির উপর নির্ভর করে	Jobs স্বাধীনভাবে কাজ করে
Fail হলে	পরের jobs আর চলে না	অন্য jobs চলতে থাকে
Progress Tracking	কঠিন	চমৎকার progress tracking আছে
ব্যবহার	Video process → Thumbnail → Notify	Bulk email, বড় file import

queue:work vs queue:listen

Command	App Load	Speed	কখন ব্যবহার
queue:work	একবার load করে	দ্রুত — Production-এ ব্যবহার করুন	Production
queue:listen	প্রতি job-এ নতুন load	ধীর — কিন্তু code change-এ restart লাগে না	Development

Task Scheduling

```
// routes/console.php (Laravel 11+)
Schedule::command('emails:send')->daily();
Schedule::command('reports:weekly')->weekly()->mondays()->at('08:00');
Schedule::job(new GenerateReport)->everyFiveMinutes();

// Server-এ মাত্র একটি Cron entry
* * * * * cd /path && php artisan schedule:run >> /dev/null 2>&1
```

৯. Real-Time: Echo, Pusher ও Reverb

WebSocket vs HTTP

বিষয়	HTTP Request-Response	WebSocket
Direction	Client → Server (one-way request)	Two-way (bidirectional)
Connection	প্রতি request-এ নতুন	একটি persistent connection
Use case	Page load, Form submit	Chat, Live notification, Real-time
Laravel Tool	Routes/Controllers	Echo + Pusher/Reverb

Pusher vs Laravel Reverb

বৈশিষ্ট্য	Pusher	Laravel Reverb
Cost	নির্দিষ্ট সীমার পর খরচ বাড়ে	সম্পূর্ণ free (নিজের server)
Speed	External server, কিছু latency	Local network — অত্যন্ত fast
Security	Data third-party server-এ যায়	Data নিজের server-এ সীমাবদ্ধ
Setup	সহজ — managed service	নিজে install করতে হয়
Scalability	Pusher handle করে	হাজার হাজার connection handle করে

১০. Stored Procedure ও Laravel Excel

Stored Procedure

সংজ্ঞা

Database-এর ভেতরে সেভ করে রাখা SQL code-এর group।
একটি Function-এর মতো — database engine-এর ভেতরেই থাকে।
বারবার complex কাজ করতে 'Call' করলেই হয়।

সুবিধা	বিবরণ
Performance	একবার compiled — সাধারণ query-র চেয়ে দ্রুত
Network Traffic	১০টি query-র বদলে একটি SP call — traffic কমে
Security	SQL Injection প্রতিরোধে সাহায্য করে
Maintainability	Database level-এ update করলেই সব জায়গায় পরিবর্তন

```
// Laravel থেকে Stored Procedure call করা
DB::statement('CALL GenerateMonthlyReport(?, ?)', [$year, $month]);

$results = DB::select('CALL GetUserOrders(?)', [$userId]);
```

Laravel Excel (Maatwebsite)

```
composer require maatwebsite/excel

// Import class তৈরি
php artisan make:import UsersImport --model=User

// UsersImport.php
class UsersImport implements ToModel, WithHeadingRow, WithValidation
{
    public function model(array $row): User
    {
        return new User([
            'name' => $row['name'],
            'email' => $row['email'],
            'password' => bcrypt($row['password']),
        ]);
    }

    public function rules(): array
    {
        return ['email' => 'required|email|unique:users'];
    }
}
```

```
// Controller-এ ব্যবহার
Excel::import(new UsersImport, $request->file('file'));
```

১১. PDF Generation (DomPDF)

Setup ও Basic Usage

```
composer require barryvdh/laravel-dompdf

use Barryvdh\DomPDF\Facade\Pdf;

public function generatePDF()
{
    $data = ['title' => 'Invoice #101', 'users' => User::all()];

    $pdf = Pdf::loadView('my_pdf_view', $data);

    return $pdf->download('invoice.pdf'); // Download
    // return $pdf->stream();             // Browser preview
}
```

চ্যালেঞ্জ	Solution
Modern CSS সাপোর্ট নেই	CSS 2.1 ব্যবহার করুন — Table/Float layout
Memory Limit Error	ini_set('memory_limit', '512M')
বাংলা (Unicode) font	meta charset=utf-8 + custom font-family declare
বড় PDF ইউজারকে wait করানো	Job Queue-এ পাঠান — email-এ link দিন
Server-এ সেভ করা	Storage::put('public/pdfs/file.pdf', \$pdf->output())

১২. OOP (Object-Oriented Programming)

OOP-এর ৪টি স্তম্ভ

 Encapsulation Data ও Methods একটি Class-এ আবদ্ধ। Access Modifiers (public/private/protected) দিয়ে data control করা হয়।	 Abstraction Complex implementation লুকিয়ে রেখে শুধু essential features প্রকাশ করা। Interfaces ও Abstract Classes ব্যবহার করা হয়।	 Inheritance Parent class-এর properties ও methods Child class-এ inherit হয়। 'is-a' relationship। Code reusability নিশ্চিত করে।	 Polymorphism একই method ভিন্ন ভিন্ন রূপে কাজ করে। Method Overriding (runtime) ও Overloading (compile-time) — দুই প্রকার।
--	--	--	--

Class ও Object

বিষয়	Class	Object
সংজ্ঞা	Blueprint / নকশা	Blueprint থেকে তৈরি বাস্তব জিনিস
মেমোরি	জায়গা দখল করে না	মেমোরিতে জায়গা নেয়
উদাহরণ	Car (নকশা)	Toyota, BMW (আসল গাড়ি)
অবজেক্ট সংখ্যা	একটি Class	একাধিক Object তৈরি করা যায়

Interface vs Abstract Class

বৈশিষ্ট্য	Interface	Abstract Class
Method body	থাকতে পারে না (সব খালি)	থাকতে পারে (concrete + abstract)
Properties	রাখা যায় না	রাখা যায়
Implement/Extend	implements keyword	extends keyword
Multiple	একটি class অনেক interface implement করতে পারে	একটি class শুধু একটি extend করতে পারে
উদ্দেশ্য	Contract / Specification define করা	Common code share করা

Composition vs Inheritance

বৈশিষ্ট্য	Inheritance	Composition
সম্পর্ক	Is-a (Dog is an Animal)	Has-a (Car has an Engine)
Coupling	Tight — parent change হলে child-এ সমস্যা	Loose — একটি change-এ অন্যটিতে প্রভাব নেই
Flexibility	Static — runtime-এ behavior বদলানো কঠিন	Dynamic — runtime-এ inject করা object বদলানো যায়
জটিলতা	Deep inheritance tree জটিল হয়	Modular ও clean code

```
// Composition উদাহরণ
class Engine {
    public function start(): string { return 'Engine started!'; }
}

class Car {
    private Engine $engine;

    // Dependency Injection — Engine inject করা হচ্ছে
```

```

public function __construct(Engine $engine) {
    $this->engine = $engine;
}

public function startCar(): string { return $this->engine->start(); }
}

$car = new Car(new Engine());
echo $car->startCar(); // Output: Engine started!

```

Anonymous Function & Closure

```

// Closure - 'use' দিয়ে বাইরের variable capture করা
$discount = 10;

$applyDiscount = function($price) use ($discount) {
    return $price - ($price * ($discount / 100));
};

echo $applyDiscount(1000); // Output: 900

// Arrow function (PHP 7.4+) - automatically captures
$applyDiscount = fn($price) => $price - ($price * ($discount / 100));

```

SQL Injection & Security Checklist

Laravel Security Checklist

- ✓ Eloquent / Query Builder ব্যবহার করুন (Raw SQL এড়িয়ে চলুন)
- ✓ User input কখনো trust করবেন না — সব কিছু validate করুন
- ✓ Mass Assignment থেকে রক্ষার জন্য \$fillable বা \$guarded ব্যবহার করুন
- ✓ CSRF token সব form-এ ব্যবহার করুন (@csrf)
- ✓ HTTPS enforce করুন production-এ
- ✓ .env file web-accessible রাখবেন না
- ✓ PDO Prepared Statement — SQL Injection সম্পূর্ণ রোধ করে

১৩. Virtual DOM & Frontend Concepts

Real DOM vs Virtual DOM

বৈশিষ্ট্য	Real DOM	Virtual DOM
আপডেট গতি	ধীর (Slow) — পুরো tree render	অনেক দ্রুত (Fast) — শুধু পরিবর্তন

বৈশিষ্ট্য	Real DOM	Virtual DOM
মেমোরি	বেশি খরচ হয়	খুব কম খরচ
রেন্ডারিং	পুরো tree আপডেট হয়	শুধু পরিবর্তিত অংশ (Patching)
ব্যবহার	Vanilla JS / jQuery	React, Vue.js
Diffing	নেই	আছে — Old vs New compare করে

Virtual DOM-এর Diffing Algorithm: ১. Data পরিবর্তন হলে পুরো Virtual DOM update হয়। ২. Old vs New Virtual DOM compare হয় (Diffing)। ৩. শুধু পরিবর্তিত অংশ খুঁজে বের করা হয়। ৪. সেই নির্দিষ্ট অংশ Real DOM-এ apply করা হয় (Reconciliation/Patching)।

Hydration

Hydration কী?

Server থেকে পাঠানো static HTML-কে browser-এ Vue/React টেকওভার করে interactive করে তোলে। এই process-কে Hydration বলে।

Server ও client-এর data আলাদা হলে 'Hydration Mismatch Error' দেখা দেয়।

Nuxt.js Concepts

বিষয়	Modules	Plugins
কখন কাজ করে	Build time	Runtime
কী করে	App configuration ও নতুন feature add	Vue global function / third-party library inject
উদাহরণ	@nuxtjs/tailwindcss	Custom chart library

বিষয়	useState	Pinia
ব্যবহার	ছোট/মাঝারি state	বড়, complex application
SSR Friendly	হ্যাঁ	হ্যাঁ
Status	Built-in Nuxt	Official recommended store
Memory leak	রোধ করে	Handle করতে হয়

বিষয়	useFetch	\$fetch
Caching	সাপোর্ট করে	করে না
SSR	হ্যাঁ	client-side only

বিষয়	useFetch	\$fetch
ব্যবহার	Page load-এর সময় data fetch	Button click বা event-এ

দ্রুত রেফারেন্স — Quick Reference

Artisan Commands

```
php artisan make:model Product -mcrf # Model+Migration+Controller+Request+Factory
php artisan make:observer OrderObserver --model=Order
php artisan make:event OrderConfirmed
php artisan make:listener SendOrderNotification --event=OrderConfirmed
php artisan make:job ProcessPayment
php artisan make:policy PostPolicy --model=Post
php artisan make:middleware EnsureTokenIsValid
php artisan queue:work --queue=high,default
php artisan schedule:run
php artisan optimize:clear # Cache clear
```

Eloquent Cheat Sheet

```
// Eager Loading
User::with(['posts', 'profile'])->get();
User::with(['posts' => fn($q) => $q->latest()])->get();
User::withCount('posts')->having('posts_count', '>', 5)->get();

// Relationships
hasOne / hasMany / belongsTo / belongsToMany
hasManyThrough / morphMany / morphToMany

// Query Scopes
User::active()->verified()->latest()->paginate(20);

// Soft Delete
User::onlyTrashed()->restore();
User::withTrashed()->find($id);
```


Redis Cache

Hydration কী?

Redis (Remote Dictionary Server) হলো একটি ওপেন-সোর্স, ইন-মেমরি ডাটা স্ট্রাকচার স্টোর। এটি ডাটাবেস, ক্যাশ এবং মেসেজ ব্রোকার হিসেবে ব্যবহৃত হয়। লারাভেলের মতো ফ্রেমওয়ার্কে পারফরম্যান্স বৃদ্ধির জন্য Redis অপরিহার্য।

Redis-এর ক্যাশিং আর্কিটেকচার (Workflow)

Redis-এর কাজের মূল ভিত্তি হলো **Cache-Aside Pattern**:

1. **Request:** অ্যাপ্লিকেশন প্রথমে Redis-এ ডেটা খোঁজে।
2. **Cache Hit:** ডেটা পাওয়া গেলে সেটি সরাসরি রিটার্ন করে।
3. **Cache Miss:** ডেটা না পাওয়া গেলে অ্যাপ্লিকেশন ডাটাবেস (DB) থেকে ডেটা তোলে।
4. **Update:** অ্যাপ্লিকেশন ডেটাবেস থেকে পাওয়া ডেটা Redis-এ সেভ করে (SET), যাতে পরবর্তী রিকোয়েস্ট দ্রুত হয়।

৩. মেমরি ম্যানেজমেন্ট ও ইভিকশন পলিসি (Eviction Policies)

যখন Redis-এর মেমরি পূর্ণ হয়ে যায়, তখন নতুন ডেটা জায়গা করার জন্য পুরনো ডেটা রিমুভ করতে হয়। Redis-এর মেমরি ম্যানেজমেন্ট অ্যালগরিদমগুলো নিচে বিস্তারিত আলোচনা করা হলো:

A. LRU (Least Recently Used)

এটি ধরে নেয় যে, যে ডেটাগুলো সম্প্রতি ব্যবহার করা হয়েছে, সেগুলো ভবিষ্যতে আবারও দরকার হবে।

- কাজের প্রক্রিয়া: Redis প্রতিটি ডেটার সাথে একটি টাইমস্ট্যাম্প রাখে। মেমরি ফুল হলে এটি সেই ডেটাগুলো ডিলিট করে যা দীর্ঘ সময় ধরে এক্সেস করা হয়নি।
- ব্যবহার: অধিকাংশ ওয়েব অ্যাপ্লিকেশনের জন্য এটি আদর্শ।

B. LFU (Least Frequently Used)

এটি ডেটার জনপ্রিয়তার ভিত্তিতে কাজ করে।

- কাজের প্রক্রিয়া: Redis প্রতিটি ডেটার ব্যবহারের সংখ্যা (Counter) ট্র্যাক করে। যে ডেটা সবচেয়ে কমবার কল করা হয়েছে, মেমরি ফুল হলে সেটি সবার আগে ডিলিট হবে।
- ব্যবহার: যেখানে নির্দিষ্ট কিছু ডেটা অনেক পুরনো হলেও বারবার প্রয়োজন হয়, সেখানে LFU কার্যকর।

C. TTL (Time To Live)

- কাজের প্রক্রিয়া: প্রতিটা কি (Key)-এর সাথে একটি নির্দিষ্ট সময় নির্ধারণ করে দেওয়া হয়। সময় শেষ হলে Redis নিজে থেকেই ডেটা মুছে ফেলে।
- অ্যালগরিদম: Redis একটি নির্দিষ্ট বিরতিতে (Sampling) কিছু রেন্ডম ডেটা চেক করে দেখে সেগুলোর TTL শেষ হয়েছে কি না।

৪. Redis অ্যালগরিদমসমূহের তুলনামূলক বিশ্লেষণ

অ্যালগরিদম	লক্ষ্য (Target)	সুবিধা	সীমাবদ্ধতা
LRU	সময় (Recency)	সাধারণ ব্যবহারের জন্য দ্রুত	সব সময় সেরা ফলাফল দেয় না
LFU	জনপ্রিয়তা (Frequency)	জনপ্রিয় ডেটা ধরে রাখে	নতুন ডেটা পুরনো হতে সময় নেয়
TTL	মেয়াদ (Expiration)	মেমরি নিয়ন্ত্রণে কার্যকর	মেয়াদ শেষ হওয়ার আগে ডেটা সরানোর ক্ষমতা নেই

৫. অ্যাডভান্সড টপিক: মেমরি পলিসি কনফিগারেশন

লারাভেল বা যেকোনো অ্যাপ্লিকেশনে আপনি **redis.conf** ফাইলে আপনার প্রয়োজন অনুযায়ী পলিসি সেট করতে পারেন:

Bash

```
# মেমরি লিমিট ২ গিগাবাইট
```

```
maxmemory 2gb
```

```
# মেমরি ফুল হলে LRU পলিসি অনুযায়ী কাজ করবে
```

```
maxmemory-policy allkeys-lru
```

পলিসিগুলো যা আপনি বেছে নিতে পারেন:

- **allkeys-lru**: যেকোনো ডেটা থেকে LRU অনুযায়ী ডিলিট করবে।
- **volatile-lru**: শুধুমাত্র মেয়াদ শেষ হতে যাওয়া (TTL) ডেটা থেকে LRU অনুযায়ী ডিলিট করবে।
- **noeviction**: মেমরি ফুল হলে নতুন ডেটা সেভ করতে বাধা দিবে (সবচেয়ে নিরাপদ)।

৬. লারাভেলে বাস্তবায়নের সেরা পদ্ধতি

লারাভেলে Redis ব্যবহারের সময় **Cache Tagging** ব্যবহার করার পরামর্শ দেওয়া হয়। এতে অনেকগুলো ক্যাশ কি (Key) একসাথে ক্যাটাগরি অনুযায়ী মুছে ফেলা যায়:

PHP

```
// লারাভেলে ট্যাগিং
```

```
Cache::tags(['products', 'active'])->put('product_1', $data, 3600);
```

```
// সব অ্যাক্টিভ প্রোডাক্ট ক্যাশ একসাথে মুছে ফেলা
```

```
Cache::tags(['active'])->flush();
```

লারাভেলে **'Scope'** ব্যবহারের সুবিধা কী?

কোড পুনরায় ব্যবহারের উপযোগী (Reusable) কুয়েরি লজিক তৈরি করা

স্কোপ ব্যবহার করে কুয়েরির অংশগুলোকে মডেলে ছোট ছোট মেথড হিসেবে রাখা যায়।

'Global Scope' এর কাজ কী?

প্রতিটি কুয়েরিতে স্বয়ংক্রিয়ভাবে একটি কন্ডিশন যোগ করা যেমন সব কুয়েরিতে 'active = 1' যোগ করার জন্য এটি আদর্শ।

Value Object

Value Object হলো ডোমেইন-ড্রিভেন ডিজাইনের (DDD) একটি গুরুত্বপূর্ণ কনসেপ্ট। সহজ কথায়, এটি এমন এক ধরনের অবজেক্ট যা কোনো নির্দিষ্ট ভ্যালু বা বৈশিষ্ট্য প্রকাশ করে, কিন্তু এর কোনো নিজস্ব পরিচয় (Identity) থাকে না।

মূল বৈশিষ্ট্য:

১. Identity নেই: দুটি Value Object-এর ভেতরের ভ্যালুগুলো যদি একই হয়, তবে আমরা তাদের একই মনে করি। এদের আলাদা কোনো আইডি (যেমন: ডাটাবেস প্রাইমারি কি) থাকে না।
২. Immutable (অপরিবর্তনীয়): একবার তৈরি হয়ে গেলে এর মান আর পরিবর্তন করা যায় না। পরিবর্তন করতে চাইলে নতুন অবজেক্ট তৈরি করতে হয়।
৩. Self-Validating: এটি নিজেকেই নিজে ভ্যালিড রাখতে পারে। যেমন: একটি Email ভ্যালু অবজেক্ট তৈরির সময় যদি ইমেইল ফরম্যাট সঠিক না হয়, তবে এটি অবজেক্ট তৈরি হতে দেবে না।

একটি উদাহরণ (সহজ ভাষায়):

ধরুন, একটি ই-কমার্স অ্যাপ্লিকেশনে Money বা Price নিয়ে কাজ করছেন।

Entity (যাদের ID আছে): User (আইডি ১), Product (আইডি ১০১)। এদের আইডি পরিবর্তন হলেও এরা একই এন্টিটি থাকে।

Value Object: Money (১০০ টাকা)। যদি আরেকটি অবজেক্টও ১০০ টাকাই হয়, তবে তাদের মধ্যে কোনো পার্থক্য নেই। এটি কোনো ডাটাবেস আইডির ওপর নির্ভর করে না।

কোড উদাহরণ (PHP/Laravel):

PHP

```
class Money {
    private int $amount;
    private string $currency;

    public function __construct(int $amount, string $currency) {
        // Validation: মান নেগেটিভ হতে পারবে না
        if ($amount < 0) {
            throw new Exception("Amount cannot be negative");
        }
        $this->amount = $amount;
        $this->currency = $currency;
    }
}
```

// ব্যবহার:

```
$price = new Money(100, 'BDT');
```

কেন এটি ব্যবহার করবেন?

কোড পরিষ্কার থাকে: বারবার ডাটাবেস ফিল্ড ভ্যালিডেশন না করে একটি ক্লাসের মাধ্যমেই ভ্যালিডেশন নিশ্চিত করা যায়।

ভুল হওয়ার সম্ভাবনা কম: যেহেতু এটি ইমিউটেবল (Immutable), তাই সিস্টেমের কোথাও ভ্যালু হঠাৎ বদলে যাওয়ার ভয় থাকে না।

রিডিবিলিটি: `new Money(100, 'BDT')` লিখলে বুঝতে সুবিধা হয় যে এটি একটি নির্দিষ্ট ভ্যালু, কোনো সাধারণ ইন্টিজার নয়।

সারকথা: যখন কোনো তথ্যের "পরিচয়" (Identity) এর চেয়ে তার "মান" (Value) বেশি গুরুত্বপূর্ণ, তখনই তাকে Value Object হিসেবে গণ্য করতে হয়।

লারাভেলে Model Event

লারাভেলে **Model Event** হলো এমন একটি ফিচার যার মাধ্যমে আপনি একটি মডেলের বিভিন্ন লাইফসাইকেল বা ঘটনার (যেমন: ডেটা ইনসার্ট হওয়া, আপডেট হওয়া, বা ডিলিট হওয়া) সময় নির্দিষ্ট কিছু কোড রান করতে পারেন।

সহজ কথায়, ডেটাবেসে কোনো পরিবর্তন হওয়ার আগে বা পরে আপনি যদি অটোমেটিক কোনো কাজ করতে চান, তবে মডেল ইভেন্ট ব্যবহার করবেন।

মডেলের প্রধান ইভেন্টসমূহ:

লারাভেল মডেলের বিভিন্ন ধাপে অটোমেটিক এই ইভেন্টগুলো ফায়ার করে:

- **retrieved**: ডাটাবেস থেকে ডেটা লোড হওয়ার পর।
- **creating** / **created**: ডেটা ইনসার্ট হওয়ার আগে / পরে।
- **updating** / **updated**: ডেটা আপডেট হওয়ার আগে / পরে।
- **saving** / **saved**: ডেটা সেভ (ক্রিয়েট বা আপডেট) হওয়ার আগে / পরে।
- **deleting** / **deleted**: ডেটা ডিলিট হওয়ার আগে / পরে।

কেন ব্যবহার করবেন?

- অটোমেশন: যেমন—একটি ইউজার রেজিস্টার করার পর অটোমেটিক তাকে একটি ওয়েলকাম ইমেইল পাঠানো (**created** ইভেন্ট)।
- ডেটা ক্লিনিং: ডেটা সেভ করার আগে নামের প্রথম অক্ষর বড় হাতের করা (**saving** ইভেন্ট)।
- লগিং: কোনো গুরুত্বপূর্ণ ডেটা আপডেট হলে তার লগ রাখা (**updated** ইভেন্ট)।

কোড উদাহরণ:

আপনি মডেলের **booted()** মেথডে ইভেন্ট ডিফাইন করতে পারেন:

PHP

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    protected static function booted()
    {
        // পোস্ট তৈরি হওয়ার আগে স্লাগ জেনারেট করা
        static::creating(function ($post) {
            $post->slug = \Str::slug($post->title);
        });

        // পোস্ট ডিলিট হওয়ার পর লগ রাখা
        static::deleted(function ($post) {
            \Log::info('Post deleted: ' . $post->id);
        });
    }
}
```

ছোট টিপস:

আপনার যদি অনেকগুলো ইভেন্ট থাকে, তবে মডেলের ভেতর সব কোড না লিখে **Observer** ব্যবহার করা সবচেয়ে ভালো। এটি মডেল এবং বিজনেস লজিকে আলাদা রাখে, ফলে কোড অনেক বেশি ক্লিন এবং মেইনটেইনেবল হয়।